

Esame 20260219

Esercizio 1

(1) Esercizio 1 v1

ESSAY marked out of 10 penalty 0 File picker

Tuo cugino ha comprato un piccolo robot con quattro ruote in grado di spostarsi in una stanza rispondendo a 5 diversi comandi: `s`: spostamento verso sud; `n`: spostamento verso nord; `e`: spostamento verso est; `o`: spostamento verso ovest; `x`: nessuno spostamento.

Tuo cugino ti chiede, viste le tue competenze informatiche, di sviluppare un programma C++ che, data una sequenza di comandi, li faccia eseguire al robot e mostri la sua traiettoria sulla mappa della stanza in cui naviga. L'utente passa al programma, da riga di comando, le dimensioni della stanza (prima viene passata la dimensione sull'asse delle ascisse, poi quella sull'asse delle ordinate), espresse come un numero intero di metri, entro i quali far navigare il robot, e una stringa contenente i comandi di navigazione di cui sopra. Per semplicità, si assuma una stanza rettangolare.

Il programma:

- Verifica che il numero di argomenti passati sia corretto; altrimenti, termina.
- Verifica successivamente che le dimensioni della stanza siano due numeri interi positivi; in caso affermativo, invoca la funzione `CreaMappa` che alloca dinamicamente la mappa (una matrice di interi); altrimenti, termina. Per semplicità, si assuma che la navigazione del robot inizi sempre dalla posizione $(0, 0)$ della matrice `mappa`.
- Per ogni comando all'interno della sequenza di comandi, si verifica se è valido, cioè se appartiene all'insieme dei comandi definiti. Se non è valido, si procede con il comando successivo e il robot non si muove. Se è valido, il programma chiama la funzione `Muovi` che verifica se il comando consente di far muovere il robot di un passo, sempre di lunghezza pari a 25 cm, nella direzione specificata dal comando. La funzione `Muovi`, inoltre, se il movimento è lecito (non comporta l'attraversamento dei muri della stanza), aggiorna anche le coordinate rispettando il comando; in caso contrario, le coordinate rimangono invariate e i successivi comandi della sequenza non vengono eseguiti.
- Se la nuova posizione è valida (verificata e calcolata, come detto al punto precedente, con la funzione `Muovi`), aggiorna nella matrice il valore dell'intero corrispondente alla nuova posizione, incrementandolo di 1 (uno). **Attenzione:** le ascisse della mappa corrispondono alle colonne della matrice, le ordinate alle righe. Se il robot non si muove, la posizione corrente viene comunque incrementata di 1 (uno). La matrice contiene quindi il percorso effettuato dal robot in risposta ai comandi.
- Al termine della sequenza di comandi (che sia stata completata o eseguita solo parzialmente), stampa la mappa con il percorso del robot nel file `Mappa.txt` e a video, quindi termina.

Le funzioni `CreaMappa` e `Muovi` devono essere implementate come segue:

- `CreaMappa`: la funzione riceve come parametri le dimensioni della stanza in metri (numeri interi positivi): il primo parametro è la dimensione sull'asse dell'ordinate (quindi le righe), il secondo parametro è la dimensione sull'asse delle ascisse (quindi le colonne). La funzione poi alloca una matrice che rappresenta la mappa e che abbia un numero di elementi tale che ogni suo elemento rappresenti un quadrato di $25\text{ cm} \times 25\text{ cm}$. La funzione inizializza ogni elemento della matrice a 0 (zero). La funzione restituisce il puntatore alla matrice che rappresenta la mappa.

- **Muovi**: la funzione riceve come parametri: (1) la posizione corrente del robot, espressa come due numeri interi x e y , entrambi passati per riferimento; (2) le dimensioni della stanza in metri (numeri interi positivi); (3) il comando corrente da fare eseguire al robot, espresso come un carattere. La funzione verifica se la mossa è valida, cioè che il robot non oltrepassi i muri. Se la mossa è valida, aggiorna la posizione del robot modificando x e y e restituisce 0 (zero); in caso contrario, restituisce -1 (meno uno), lasciando x e y invariate.

Non si utilizzino librerie diverse rispetto a quelle indicate! Pena annullamento dell'esercizio!

Non si implementino altre funzioni tranne quelle richieste! Pena annullamento dell'esercizio!

Di seguito è riportato un esempio di esecuzione del programma.

```
computer > ./a.out 2 2 "ssseessxxnn"
1 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0
1 1 2 0 0 0 0 0
0 0 2 0 0 0 0 0
0 0 3 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0

computer > ./a.out 2 3 "eesseektssooooookkssoo"
Mossa non valida, esco dal programma
1 1 1 0 0 0 0 0
0 0 1 0 0 0 0 0
0 0 1 1 3 0 0 0
0 0 0 0 1 0 0 0
1 1 1 1 1 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
```

Note:

- **In caso di errori di compilazione della parte fornita, segnalare tempestivamente il problema al docente in aula per trovare una soluzione comune a tutti.**
- Scaricare i file `esercizio1.cpp`, modificare il solo file `esercizio1.cpp` per inserire il codice necessario per rispondere a questo esercizio. **Caricare il file sorgente risultato delle vostre modifiche a soluzione di questo esercizio** nello spazio apposito.
- All'interno di questo programma **non è ammesso** l'utilizzo di variabili globali o di tipo `static` e di funzioni di libreria al di fuori di quelle definite in `iostream` e `cstdlib`.
- Si ricorda che, gli esempi di esecuzione sono puramente indicativi, e la soluzione proposta **NON** deve funzionare solo per l'input fornito, ma deve essere robusta a variazioni compatibili con la specifica riportata in questo testo.
- Si ricorda di inserire solo nuovo codice e di **NON MODIFICARE** il resto del programma (pena annullamento dell'esercizio).
- Si ricorda che la soluzione deve essere implementata in C++ **NON usando altri elementi della C++ standard template library** tranne le funzioni definite nelle librerie concesse, anche se il file compila senza cambiare gli header!

[esercizio1.cpp](#)

Information for graders:

(2) Esercizio 1 v2

ESSAY

marked out of 10

penalty 0

File picker

Tuo cugino ha comprato un piccolo robot con quattro ruote in grado di spostarsi in una stanza rispondendo a 5 diversi comandi:

- a: spostamento verso l'alto
- b: spostamento verso il basso
- d: spostamento verso destra
- s: spostamento verso sinistra
- x: nessuno spostamento

Tuo cugino ti chiede, viste le tue competenze informatiche, di sviluppare un programma C++ che, data una sequenza di comandi, li faccia eseguire al robot e mostri la sua traiettoria sulla mappa della stanza in cui naviga. L'utente passa al programma, da riga di comando, le dimensioni della stanza (prima viene passata la dimensione sull'asse delle ascisse, poi quella sull'asse delle ordinate), espresse come un numero intero di metri, entro i quali far navigare il robot, e una stringa contenente i comandi di navigazione di cui sopra. Per semplicità, si assuma una stanza rettangolare.

Il programma:

- Verifica che il numero di argomenti passati sia corretto; altrimenti, termina.
- Verifica successivamente che le dimensioni della stanza siano due numeri interi positivi; in caso affermativo, invoca la funzione `Mappa` che alloca dinamicamente la mappa (una matrice di interi); altrimenti, termina. Per semplicità, si assuma che la navigazione del robot inizi sempre dalla posizione $(0, 0)$ della matrice `mappa`.
- Per ogni comando all'interno della sequenza di comandi, si verifica se è valido, cioè se appartiene all'insieme dei comandi definiti. Se non è valido, si procede con il comando successivo e il robot non si muove. Se è valido, il programma chiama la funzione `Mossa` che verifica se il comando consente di far muovere il robot di un passo, sempre di lunghezza pari a 25 cm, nella direzione specificata dal comando. La funzione `Mossa`, se il movimento è lecito (non comporta l'attraversamento dei muri della stanza), aggiorna anche le coordinate rispettando il comando; in caso contrario, le coordinate rimangono invariate e i successivi comandi della sequenza non vengono eseguiti.
- Se la nuova posizione è valida (verificata e calcolata, come detto al punto precedente, con la funzione `Mossa`), aggiorna nella matrice il valore dell'intero corrispondente alla nuova posizione, incrementandolo di 1 (uno). **Attenzione:** le ascisse della mappa corrispondono alle colonne della matrice, le ordinate alle righe. Se il robot non si muove, la posizione corrente viene comunque incrementata di 1 (uno). La matrice contiene quindi il percorso effettuato dal robot in risposta ai comandi.
- Al termine della sequenza di comandi (che sia stata completata o eseguita solo parzialmente), stampa la mappa con il percorso del robot nel file `Mappa.txt` e a video, quindi termina.

Le funzioni `Mappa` e `Mossa` siano implementate come segue:

- Mappa:** la funzione riceve come parametri le dimensioni della stanza in metri (numeri interi positivi): il primo parametro è la dimensione sull'asse dell'ordinate (quindi le righe), il secondo parametro è la dimensione sull'asse delle ascisse (quindi le colonne). La funzione poi alloca una matrice che rappresenta la mappa e che abbia un numero di elementi tali che ogni suo elemento rappresenti un quadrato di $25\text{ cm} \times 25\text{ cm}$. La funzione inizializza ogni elemento della matrice a 0 (zero). La funzione restituisce il puntatore alla matrice che rappresenta la mappa.

- **Mossa:** la funzione riceve come parametri: (1) la posizione corrente del robot, espressa come due numeri interi x e y , entrambi passati per riferimento; (2) le dimensioni della stanza in metri (numeri interi positivi); (3) il comando corrente da fare eseguire al robot, espresso come un carattere. La funzione verifica se la mossa è valida, cioè che il robot non oltrepassi i muri. Se la mossa è valida, aggiorna la posizione del robot modificando x e y e restituisce 0 (zero); in caso contrario, restituisce -2 (meno due), lasciando x e y invariate.

Non si utilizzino librerie diverse rispetto a quelle indicate! Pena annullamento dell'esercizio!

Non si implementino altre funzioni tranne quelle richieste! Pena annullamento dell'esercizio!

Di seguito è riportato un esempio di esecuzione del programma.

```
computer > ./a.out 2 3 "xxdddbbbbssysa"
3 1 1 1 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 1 0 0 0 0
1 0 0 1 0 0 0 0
1 2 1 1 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
computer > ./a.out 2 2 "ddbbbttyudddaaaaa"
Mossa non valida, esco dal programma
1 1 1 0 0 1 0 0
0 0 1 0 0 1 0 0
0 0 1 0 0 1 0 0
0 0 4 1 1 1 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
```

Note:

- In caso di errori di compilazione della parte fornita, segnalare tempestivamente il problema al docente in aula per trovare una soluzione comune a tutti.
- Scaricare i file `esercizi01.cpp`, modificare il solo file `esercizi01.cpp` per inserire il codice necessario per rispondere a questo esercizio. **Caricare il file sorgente risultato delle vostre modifiche a soluzione di questo esercizio** nello spazio apposito.
- All'interno di questo programma **non è ammesso** l'utilizzo di variabili globali o di tipo `static` e di funzioni di libreria al di fuori di quelle definite in `iostream` e `cstdlib`.
- Si ricorda che, gli esempi di esecuzione sono puramente indicativi, e la soluzione proposta **NON** deve funzionare solo per l'input fornito, ma deve essere robusta a variazioni compatibili con la specifica riportata in questo testo.
- Si ricorda di inserire solo nuovo codice e di **NON MODIFICARE** il resto del programma (pena annullamento dell'esercizio).
- Si ricorda che la soluzione deve essere implementata in C++ **NON usando altri elementi della C++ standard template library tranne le funzioni definite nelle librerie concesse, anche se il file compila senza cambiare gli header!**

[esercizi01.cpp](#)

Information for graders:

Total of marks: 20